

TCS NQT On-Campus Coding Workout Sheets

Topic-Wise Track Separations: Clean Problem Formulation & Unseen Test Cases

Array & Mathematics

The Ritual of Circular Kriya

12th November Shift 1 — Q1

Problem Description

In an ancient spiritual tradition, there are **N** sacred energy centers arranged in a perfect circular formation, numbered sequentially from 1 to N. Center 1 is adjacent to Center 2, Center 2 is adjacent to Center 3, and due to the circular nature, Center N is adjacent to Center 1.

A monk practices a specialized technique called "Kriya". When a monk performs Kriya on a specific energy center X, he absorbs the energy value of that center (which is equal to its number, X). However, performing Kriya comes with a significant cost: **the center X itself, its immediate predecessor (X-1), and its immediate successor (X+1) all vanish instantly.**

Once a center has vanished, it can no longer be used to perform Kriya, nor can its energy be absorbed. The monk wants to perform a series of Kriyas on different centers to maximize the total absorbed energy. Furthermore, there is a Multiplication Factor **M** provided for the ritual. The final result of the ritual is the maximum possible energy sum multiplied by M.

Your task is to calculate the final spiritual energy result for the given N and M.

CONSTRAINTS

- $3 \leq N \leq 10^9$ (Note: For $N < 3$, the neighbors overlap significantly)
- $1 \leq M \leq 100$

I/O FORMAT

Input: A single line containing two space-separated integers, N and M.

Output: A single integer representing the maximum energy sum multiplied by M.

Example Test Cases

Input :

5 4

Output :

32

Explanation:

Energy centers are [1, 2, 3, 4, 5] arranged in a circle.

- If the monk performs Kriya on center 5, centers 4, 5, and 1 vanish. (Current Sum = 5). Remaining centers: [2, 3].
- Now, he can perform Kriya on center 3. Center 2 vanishes. (New Sum = 5 + 3 = 8).
- Total Maximum Sum = 8.
- Final Result = $8 \times M = 8 \times 4 = 32$.

Input :

10 4

Output :

120

Explanation:

Energy centers are [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] in a circle.

To maximize the sum while ensuring no selected centers are neighbors, perform Kriya on centers: 10, 8, 6, 4, and 2.

- Total Maximum Sum = $10 + 8 + 6 + 4 + 2 = 30$.
- Final Result = $30 \times 4 = 120$.

Maximum GCD Pair with Maximum Absolute Difference

12th November Shift 2 — Q2

Problem Description

You are given two integer arrays, A and B, both of size N. Your task is to select exactly one element **a** from array A and one element **b** from array B such that their Greatest Common Divisor, **gcd(a, b)**, is as large as possible.

If there are multiple pairs (a, b) that result in the same maximum GCD value, you must break the tie by choosing the pair that yields the **largest absolute difference** **|a - b|**.

Output two values: the maximum GCD found and the maximum absolute difference associated with that specific GCD value.

CONSTRAINTS

- $1 \leq N \leq 10^5$

I/O FORMAT

Input: Line 1: An integer N representing array sizes.
Line 2: N space-separated integers representing

$$1 \leq A[i], B[i] \leq 10^6$$

array A.

Line 3: N space-separated integers representing array B.

Output: Two space-separated integers: the Maximum GCD and the Maximum Absolute Difference for that specific GCD.

Example Test Cases

Input :

3

12 18 6

18 30 10

Output :

18 0

Explanation:

The maximum common divisor across pairs is 18, which comes from choosing 18 from A and 18 from B. Their absolute difference is $|18 - 18| = 0$.

Input :

2

12 24

18 36

Output :

12 24

Explanation:

Pairs with GCD 12 are:

• (12, 36) → GCD: 12, Diff: $|12 - 36| = 24$

• (24, 36) → GCD: 12, Diff: $|24 - 36| = 12$

The maximum GCD is 12, and the maximum absolute difference for it is 24.

String Manipulation

Decrypting the King's Secret Message

12th November Shift 1 — Q2

Problem Description

A King has encrypted a secret message using a simple substitution-addition cipher. To decrypt it, you must first identify the correct Key from a list of potential candidates based on the following rules:

1. Each letter of the alphabet is assigned a fixed numerical value: A=1, B=2, C=3, ..., Z=26.

2. A Key is chosen, which is a single uppercase English character. Let the value of this key be K.
3. Every character in the original message is transformed into an integer by adding K to its original alphabet value.
4. The final encrypted message is a sequence of integers separated by spaces.

You are given a string containing several potential candidate keys and the encrypted message (a sequence of integers). The correct key is the **first character** in the candidate string that satisfies the following condition: when its value is subtracted from every integer in the encrypted message, the resulting values must **all be valid alphabet positions** (integers between 1 and 26 inclusive).

Once the correct key is found, use it to decrypt the integers back into their corresponding uppercase letters and output the original message.

CONSTRAINTS

- The candidate key string will only contain uppercase English letters.
- The encrypted message will consist of positive integers.
- There is guaranteed to be at least one valid key in the candidate string.
- $1 \leq \text{Length of Message} \leq 10^5$
- $1 \leq \text{Number of Candidates} \leq 26$

I/O FORMAT

Input: First Line: A string representing a list of potential candidate keys (e.g., "AYBTHC").

Second Line: A sequence of space-separated integers representing the encrypted message.

Output: A single string representing the decrypted original message.

Example Test Cases

Input :

AYBTHC

28 25 31 31 35

Output :

HEKKO

Explanation:

Let's test candidates one by one:

1. Candidate 'A' (Value 1): $28 - 1 = 27$ (Invalid, greater than 26).
2. Candidate 'Y' (Value 25): $25 - 25 = 0$ (Invalid, less than 1).
3. Candidate 'B' (Value 2): $28 - 2 = 26$ (Valid), but $35 - 2 = 33$ (Invalid).
4. Candidate 'T' (Value 20):
 - $28 - 20 = 8 \rightarrow \text{'H'}$
 - $25 - 20 = 5 \rightarrow \text{'E'}$
 - $31 - 20 = 11 \rightarrow \text{'K'}$
 - $31 - 20 = 11 \rightarrow \text{'K'}$
 - $35 - 20 = 15 \rightarrow \text{'O'}$

All results are within $[1, 26]$. 'T' is the correct key.

Input :

XYZAB

10 11 12

Output :

IJK

Explanation:

1. Candidates 'X' (24), 'Y' (25), 'Z' (26) result in negative positions or 0. Invalid.

2. Candidate 'A' (Value 1):

- $10 - 1 = 9 \rightarrow 'I'$
- $11 - 1 = 10 \rightarrow 'J'$
- $12 - 1 = 11 \rightarrow 'K'$

All results are valid, so 'A' is the correct key.

The Mirror Chronicle

12th November Shift 2 — Q1

Problem Description

In the subterranean chambers of the Imperial Memory Vault, historians preserve a legendary manuscript known as the Mirror Chronicle. The Chronicle is written as a single uninterrupted string of lowercase characters. Ancient scholars believed that hidden inside every Chronicle lies a Mirror Fragment — a contiguous segment of text that reads identically forward and backward (a palindrome).

The Emperor has issued a decree: From the given Chronicle, identify the **longest contiguous Mirror Fragment**. If multiple fragments share the same maximum length, return the one that **appears first** in the Chronicle.

CONSTRAINTS

- $1 \leq \text{Length of string} \leq 1000$
- The string consists only of lowercase English letters.

I/O FORMAT

Input: A single line containing a string of lowercase letters representing the Chronicle.

Output: The longest palindromic substring. If multiple palindromes have the same maximum length, output the one that occurs first.

Example Test Cases

Input :

babad

Output :

bab

Explanation:

"bab" is a palindrome of length 3. "aba" is also a palindrome of length 3, but "bab" appears first in the string, so it is preferred.

Input :

forgeeksskeegfor

Output :

geeksskeeg

Explanation:

The substring "geeksskeeg" is a perfectly symmetrical palindrome of length 10, which is the longest possible in the string.

Matrix & Grid Operations

Minimum Swaps to Center

13th November Shift 1 — Q2

Problem Description

You are given an $N \times N$ matrix of integers, where N is always an **odd integer**. Your task is to find the maximum element in the matrix and move it to the exact center cell of the matrix. The center of an $N \times N$ matrix is defined as the cell at row $\lfloor N/2 \rfloor$ and column $\lfloor N/2 \rfloor$ (using 0-indexing).

To move the element, you can only perform swaps between adjacent rows or adjacent columns. Each such swap counts as one move. Find the minimum number of swaps required to place the maximum element at the center.

Note: If there are multiple occurrences of the maximum value, choose the one that requires the absolute minimum number of swaps to reach the center.

CONSTRAINTS

- N is an odd integer ($1 \leq N \leq 1001$).
- Matrix elements are integers in the range $[-10^9, 10^9]$.

I/O FORMAT

Input: Line 1: An odd integer N .

Next N lines: Each contains N space-separated integers representing matrix rows.

Output: An integer representing the minimum number of swaps required.

Example Test Cases

Input :

```
3
1 2 3
4 9 5
6 7 8
```

Output :

```
0
```

Explanation:

The maximum element is 9, located at index (1, 1). The center of a 3×3 matrix is also (1, 1). Since it is already at the center, 0 swaps are needed.

Input :

```
5
0 0 0 0 0
0 0 0 0 0
0 25 0 0 0
0 0 0 0 0
0 0 0 0 0
```

Output :

```
1
```

Explanation:

The maximum element is 25, located at (2, 1). The center of a 5×5 matrix is (2, 2). Minimum swaps = $|2 - 2| + |1 - 2| = 0 + 1 = 1$.

Input :

```
3
10 1 2
3 4 5
6 7 10
```

Output :

```
2
```

Explanation:

The maximum value is 10, appearing at both (0, 0) and (2, 2). The center is (1, 1).

- Distance from (0, 0) to (1, 1) = $|0 - 1| + |0 - 1| = 2$ swaps.
- Distance from (2, 2) to (1, 1) = $|2 - 1| + |2 - 1| = 2$ swaps.

The minimum distance across ties is 2.

Bit Manipulation

Maximize XOR Sum with K-Set Bits

20th November — Q1

Problem Description

You are given an array of N integers and an integer K . Your task is to find a non-negative integer X such that:

1. X has exactly K set bits (1s) in its binary representation.
2. The total sum S is maximized, where $S = \sum (X \text{ XOR } \text{arr}[i])$ for i from 0 to $N-1$.

If there are multiple values of X that result in the same maximum sum, you must return the **minimum possible value of X** .

CONSTRAINTS

- $1 \leq N \leq 10^5$
- $0 \leq \text{arr}[i] \leq 10^9$
- $1 \leq K \leq 30$
- X is a non-negative 31-bit integer ($0 \leq X < 2^{31}$)

I/O FORMAT

Input: Line 1: An integer N .

Line 2: N space-separated integers representing the elements of the array.

Line 3: An integer K representing the required set bits count.

Output: A single integer representing the minimum value of X that maximizes the total XOR sum.

Example Test Cases

Input :

```
3
1 2 3
1
```

Output :

```
1073741824
```

Explanation:

$K = 1$. Looking across bits up to the 30th bit: the highest bits (like $2^{30} = 1073741824$) have three 0s and zero 1s in the array elements. Flipping this bit via XOR provides a monumental gain of 3×2^{30} to the total sum, which outweighs any lower bit variations. Hence, setting the 30th bit yields the optimal minimum X .

Dynamic Programming

Minimum N-Digit Perfect Square Sum Number

Problem Description

Given an integer N signifying a count of digits, your task is to find the **minimum possible N-digit number** such that:

1. The sum of the squares of its digits forms a perfect square.
2. The digit '0' is completely excluded from the number.

CONSTRAINTS

- $1 \leq N \leq 10^6$
- Time Limit: 1.0 second

I/O FORMAT

Input: A single integer N indicating the number of digits.

Output: A string representing the smallest lexicographical N -digit number satisfying the conditions.

Example Test Cases

Input :

1

Output :

1

Explanation:

$1^2 = 1$, which is a perfect square. Digit '0' is absent.

Input :

2

Output :

34

Explanation:

$3^2 + 4^2 = 9 + 16 = 25$ (5^2). 34 is the smallest valid 2-digit number.

Input :

3

Output :

122

Explanation:

$1^2 + 2^2 + 2^2 = 1 + 4 + 4 = 9$ (3^2). This is the smallest valid 3-digit combination.

Input :

4

Output :

1111

Explanation:

$1^2 + 1^2 + 1^2 + 1^2 = 4 (2^2)$. Smallest 4-digit number without 0s.

Greedy & Simulation

Kitchen Task Scheduler

13th November Shift 2 — Q1

Problem Description

You are a chef in a high-paced kitchen. You are given a list of tasks to complete, where each task is represented by a character string (e.g., 'A', 'B', 'C') indicating a specific dish. Every dish takes exactly 1 unit of time to cook. However, there is a constraint: after cooking a specific dish, you must wait for a cooldown period of **N** time units before you can cook the exact same dish again.

During this cooldown period, you can cook other available dishes from your list. If no other dish is available to cook during a particular time unit (because all remaining tasks are either completed or currently in their cooldown period), you must remain idle for that unit of time.

Your goal is to complete all tasks in the minimum amount of time possible and return the full chronological sequence of dishes cooked, using the string **"NOTHING"** for every time unit spent idling.

CONSTRAINTS

- $1 \leq \text{Number of tasks} \leq 10^5$
- $0 \leq \text{Cooldown factor } N \leq 100$

I/O FORMAT

Input: Line 1: A space-separated sequence of characters representing the tasks array.

Line 2: An integer representing the cooldown period **N**.

Output: A printed string array matching the precise order of execution, including **"NOTHING"**.

Example Test Cases

Input :

A A A B B B
2

Output :

['A', 'B', 'NOTHING', 'A', 'B', 'NOTHING', 'A', 'B']

Explanation:

N=2. Cooldown timeline:

- Time 0: Cook 'A'. Cooldown for 'A' lasts for Time 1 and 2.
- Time 1: Cook 'B'. Cooldown for 'B' lasts for Time 2 and 3.
- Time 2: Both are in cooldown, so print "NOTHING".
- Time 3: 'A' is ready. Cook 'A'. Cooldown for 'A' blocks Time 4, 5.
- Time 4: 'B' is ready. Cook 'B'. Cooldown for 'B' blocks Time 5, 6.
- Time 5: Both blocked, print "NOTHING".
- Time 6, 7: Cook 'A', then 'B' as they open up. Total time units = 8.

Input :

A B C A B C
3

Output :

['A', 'B', 'C', 'NOTHING', 'A', 'B', 'C']

Explanation:

After cooking A, B, and C sequentially, at Time 3, 'A' is still on cooldown until Time 4. No other task is remaining or ready, so an idle unit ("NOTHING") is injected.

Optimal Yearly Project Selection

20th November — Q2

Problem Description

You are managing industrial projects over a duration of L years. For each year, you are given a maximum capital budget and a minimum required production level. There are N available projects you can choose from.

Constraints for each year i :

1. You can select at most one project in any given year.
2. A project j can only be selected in year i if:
 - The project's Capital Cost is less than or equal to the year's budget (C_i).
 - The project's Production Value is greater than or equal to the year's requirement (P_i).

3. Projects can be reused across different years (i.e., selecting a project in Year 1 does not deplete it or prevent it from being selected in Year 2).

Given a 2D array `NPV[L][N]`, where `NPV[i][j]` is the profit of project `j` if performed in year `i`, find the maximum possible total NPV summed over all `L` years.

CONSTRAINTS

- $1 \leq L, N \leq 500$
- $1 \leq \text{Budgets}[i], \text{ProjectCosts}[i] \leq 10^9$
- $1 \leq \text{Requirements}[i], \text{ProjectProductions}[i] \leq 10^9$

I/O FORMAT

Input: Line 1: Two integers `L` and `N`.

Line 2: `L` integers for yearly budgets.

Line 3: `L` integers for yearly production requirements.

Line 4: `N` integers for capital cost of each project.

Line 5: `N` integers for production value of each project.

Next `L` lines: Each containing `N` integers representing rows of the NPV grid.

Output: An integer representing the maximum total NPV summed over all `L` years.

Example Test Cases

Input :

`L = 2, N = 3`

`Budgets = [20, 15]`

`Requirements = [25, 30]`

`ProjectCosts = [25, 21, 14]`

`ProjectProductions = [20, 26, 31]`

NPV Table:

`10 15 12`

`8 18 20`

Output :

`32`

Explanation:

- Year 1 (Budget ≤ 20 , Production ≥ 25): Project 1 and 2 exceed budget. Project 3 is valid (Cost 14, Production 31). NPV gained = 12.
- Year 2 (Budget ≤ 15 , Production ≥ 30): Project 3 is the only valid option (Cost 14, Production 31). NPV gained = 20.
- Total Max NPV = 12 + 20 = 32.